

POSTORDER TRAVERSAL (NON-RECURSIVE STACK IMPLEMENTATION)

Exact Original Code Structure with Line-by-Line Explanatory Comments

```
#include <stdio.h> // Standard I/O library for printf/scanf
void push(int);    // Function prototype for push
int pop();         // Function prototype for pop
int stack[100];    // Array-based auxiliary stack to store tree indices
int top = -1;      // Top of stack initialized to -1 (empty stack)
void push(int a)   // Push function definition
{
    top++;          // Increment stack top
    stack[top] = a; // Store node index 'a' at top of stack
}
int pop()          // Pop function definition
{
    int a = stack[top]; // Retrieve element from top of stack
    top--;             // Decrement top index
    return a;          // Return popped node index
}
int main()         // Main entry point
{
    int n;          // Variable for array/tree capacity
    printf("Enter the number of elements in the binary tree\n"); // Prompt for total node count
    scanf("%d", &n); // Read total elements count
    int a[n];        // Array representation of binary tree
    printf("Enter the elements\n"); // Prompt for tree node values
    for (int i = 0; i < n; i++) // Loop to input all elements
        scanf("%d", &a[i]); // Store tree nodes in array (-1 indicates NULL)
    int i = 0;        // Current node pointer initialized to root (0)
    int lastVisited = -1; // Tracks previously processed node index to handle
    backtracking
    while (i < n || top != -1) // Continue until all nodes visited and stack empty
    {
        while (i < n && a[i] != -1) // Traverse down to the leftmost valid descendant
        {
            push(i); // Push current ancestor index to stack
            i = 2*i + 1; // Move to left child index (2*i + 1)
        }
        if (top != -1) // If stack has pending ancestor nodes
        {
            int current = stack[top]; // Peek current top node without popping
            int right = 2*current + 2; // Calculate right child index (2*current + 2)
            if (right < n && a[right] != -1 && lastVisited != right) // If right child exists and is unvisited
            {
                i = right; // Move traversal pointer to right subtree
            }
        }
        else { // Both subtrees processed, ready to visit root
            i = pop(); // Pop node from stack
            printf("%d->", a[i]); // Visit node (Postorder: Left -> Right -> Root)
            lastVisited = i; // Mark current node as last visited
            i = n; // Set i out of bounds to trigger next pop in next cycle
        }
    }
    return 0; // Program finished successfully
}
```